Lezione 2 - Sintassi e costrutti di base

Introduzione

Nella lezione precedente abbiamo visto una panoramica su **Java**, le sue caratteristiche principali e i motivi per cui, dagli anni ’90 a oggi, è diventato uno dei linguaggi più utilizzati.

Ora ci concentreremo sul **linguaggio vero e proprio**, analizzando la sua sintassi e i costrutti di base.

Sintassi e costrutti di base

Come tutti i linguaggi di programmazione, anche Java segue precise **regole di sintassi**.
Rispetto a Python, Java ha uno stile più simile al **C** ed è un linguaggio **fortemente tipizzato:**

- **significa che ogni variabile e ogni funzione devono dichiarare esplicitamente il tipo di dati che utilizzano.**

In altre parole, Java richiede maggiore rigore rispetto a Python, dove è possibile scrivere codice in modo più flessibile. Questa struttura più rigorosa permette però di prevenire molti errori a tempo di compilazione e rende il codice più chiaro e sicuro.

Nei paragrafi seguenti esploreremo i principali **costrutti del linguaggio Java**, partendo dalle basi fino alle funzionalità più avanzate.

Il metodo `main` in Java

A differenza di **Python**, dove è possibile scrivere codice direttamente nel file senza doverlo inserire in una funzione specifica, in **Java il metodo `main` rappresenta l’entry point della JVM**.
Questo significa che l’esecuzione di un programma Java inizia sempre dal metodo `main`.

Un esempio di struttura di base di un programma Java:

Java

```
1  public class Prova {
```

```java
    public static void main(String[] args) {
        // istruzione1
        // istruzione2
    }

    // altre funzioni
    funzione1();
    funzione2();
}
```

Alcuni punti importanti:

- Tutte le istruzioni e le funzioni devono essere **definite all'interno di una classe**, perché in Java non esistono funzioni libere come in Python.
- Il metodo `main` deve avere **questa esatta firma**: `public static void main(String[] args)`
  - `public`: **indica che il metodo è accessibile alla JVM dall'esterno**.
  - `static`: **significa che il metodo può essere eseguito senza creare un'istanza della classe**.
  - `void`: **indica che il metodo non restituisce valori**.
  - `String[] args`: **permette di passare eventuali argomenti da linea di comando**.

In sintesi, il metodo `main` è fondamentale per avviare un programma Java e tutte le istruzioni devono essere strutturate all'interno della classe a cui appartengono.

Stile di codifica in Java

In Java, lo **stile di codifica** è importante per rendere il codice leggibile e comprensibile, sia per il compilatore sia per chi legge il programma.
Alcune regole fondamentali sono:

- **Spaziature**:
  - **gli elementi del codice devono essere separati da almeno uno spazio, per migliorare la leggibilità**.
- **Terminazione delle istruzioni**:
  - **ogni istruzione deve terminare con un punto e virgola** `;`.
- **Organizzazione del codice**:

- **anche se è possibile scrivere più istruzioni sulla stessa riga, è buona pratica mettere una sola istruzione per riga, così il codice risulta più chiaro e facile da mantenere**.

Seguire queste regole non è obbligatorio per la compilazione, ma aiuta a scrivere codice più ordinato e professionale, soprattutto in progetti condivisi con altri sviluppatori.

Commenti in Java

In Java esistono diversi tipi di commenti, utilizzati per **annotare il codice** senza influenzarne l'esecuzione:

1. **Commenti su una singola riga**
   - Sintassi: `//`
   - Tutto il testo che segue `//` sulla stessa riga viene ignorato dal compilatore.

Java

```java
// Questo è un commento su una riga
```

2. **Commenti multilinea**

- Sintassi: `/* ... */`
- Permette di commentare più righe contemporaneamente.

Java

```java
/* Questo è un commento
   su più righe */
```

3. **Commenti Javadoc**

- Sintassi: `/** ... */`
- Servono a generare **documentazione HTML standard** tramite lo strumento `javadoc.exe`.
- Consentono di aggiungere **tag speciali** per descrivere classi, metodi e parametri.

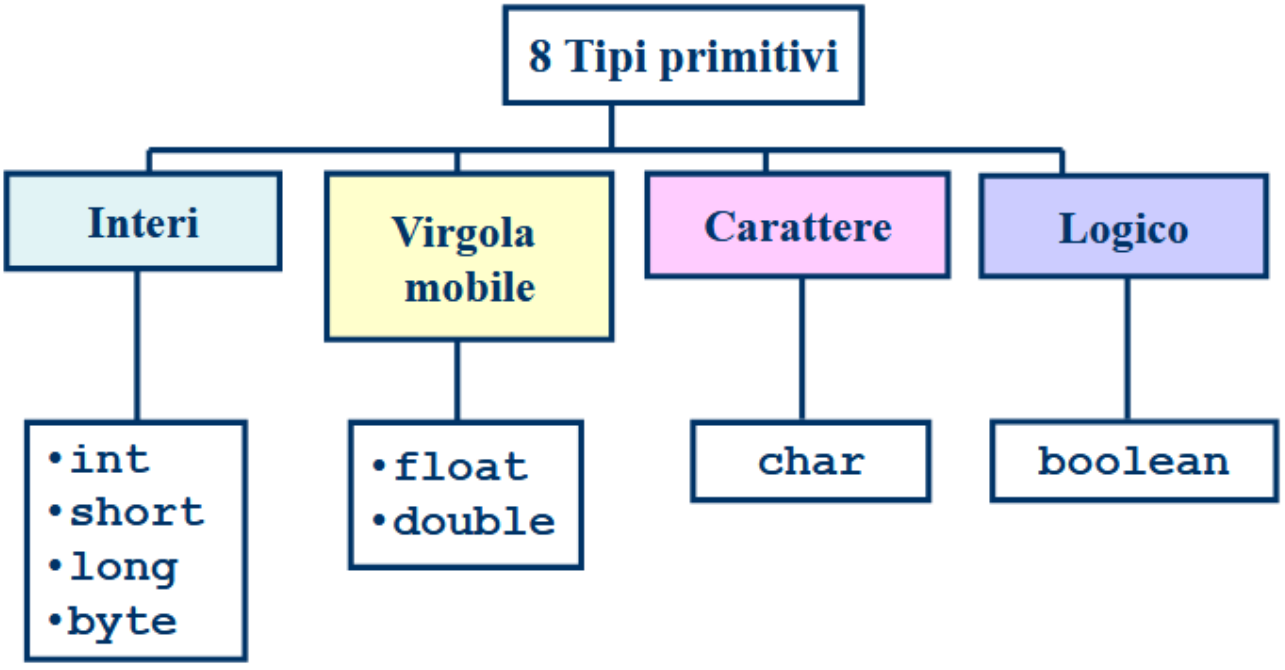
Java

```java
/**
 * Questa è una classe di esempio.
 * @author Marco
 * @version 1.0
 */
public class Esempio {
    // codice della classe
}
```

I commenti Javadoc sono particolarmente utili per creare documentazione leggibile e professionale, conforme agli standard Java, senza modificare il comportamento del programma.

Tipi di dato primitivi

In Java i **tipi primitivi** rappresentano i dati di base del linguaggio.
A differenza di Python dove **ogni dato è un oggetto**, in Java i tipi di dato primitivi non sono oggetti ma anche qui e costituiscono i mattoni fondamentali per costruire variabili ed espressioni.
Java mette a disposizione **8 tipi primitivi**, che possono essere raggruppati in **4 macro-categorie**, in base alla natura del dato rappresentato.



1. **Interi**:
   - Comprendono: `byte`, `short`, `int`, `long`
   - Permettono di memorizzare **numeri interi** (senza parte decimale).
   - Si differenziano per lo **spazio di memoria occupato** e per l'intervallo di valori rappresentabili.

- Sono tutti **con segno** (signed).

Tipo	Dimensione
`byte`	1 byte
`short`	2 byte
`int`	4 byte
`long`	8 byte

- `int` è il tipo intero più utilizzato di default.

2. **Virgola mobile**:
   Comprendono: `float`, `double`
   - **Permettono di memorizzare numeri reali, cioè con la parte decimale.**
   - **Sono utilizzati per calcoli scientifici o valori approssimati**.
   - Differiscono per **precisione** e **consumo di memoria**:

Tipo	Dimensione	Precisione
`float`	4 byte	minore
`double`	8 byte	maggiore

- `double` è il tipo predefinito per i numeri decimali.

3. **Carattere**:
   - Comprende: `char`
   - Memorizza **un singolo carattere Unicode**.
   - Occupa **2 byte**, poiché Java utilizza la codifica Unicode (UTF-16).
   - Può rappresentare lettere, numeri, simboli e caratteri speciali.

Java

```java
char lettera = 'A';
```

4. **Logico**:
   - Comprende: `boolean`
     - Può assumere **solo due valori**: `true` o `false`.
     - È utilizzato per:

- condizioni (`if` , `while` , `for`)
- controllo del flusso di esecuzione
- espressioni logiche
  Esempio:

Java

```java
boolean valido = true;
```

Questi tipi primitivi sono alla base della programmazione in Java e vengono spesso combinati per creare strutture dati più complesse o per definire variabili nelle classi.

Tipi Interi (`int` , `short` , `long` , `byte`):

I tipi interi in Java permettono di memorizzare **numeri senza decimali**. Esistono diverse varianti per ottimizzare l'uso della memoria a seconda delle esigenze del programma:

Tipo	Requisiti	Intervallo(inclusivo)
int	4 byte	Da -2.147.483.648 a +2.147.483.648
short	2 Byte	Da -32.768 a +32.768
long	8 byte	- - 9.223.372.036.854.775.808L a + + 9.223.372.036.854.775.807L
byte	1byte	Da -128 a +127

Nota: il suffisso `L` indica che il numero è di tipo `long` .

Tipi in virgola mobile (`float` , `double`):

I tipi a virgola mobile servono per memorizzare **numeri con decimali**, utili per calcoli scientifici o approssimazioni.
Java distingue tra precisione singola (`float`) e doppia (`double`):

Tipo	Requisiti	Intervallo(inclusivo)
float	4 byte	circa +- 3.40282347E+38 F (6-7 cifre decimali significative)

Tipo	Requisiti	Intervallo(inclusivo)
double	8 byte	circa +- 1.79769313486231570E+308 (15 cifre decimali significative)

> **Nota: il suffisso `F` indica che il numero è di tipo `float`.**

> **In Java essendoci questa distinzione tra `float` e `double` per numeri decimali si utilizza per lo più il tipo `double` (in Python utilizzavamo solo il `float`, anche perché per rappresentare e memorizzare i numeri decimali esiste solo il tipo `float`)**

Tipo carattere (`char`):

Il tipo `char` in Java rappresenta un **singolo carattere Unicode**, permettendo di memorizzare qualsiasi simbolo o lettera esistente.

- **Ogni `char` occupa 2 byte in memoria.**
- I caratteri si scrivono tra **apici singoli**:

Java

```java
char c1 = 'A';
char c2 = 'a';
char nullo = '\0';      // carattere nullo
char nulloUnicode = '\u0000';  // stesso valore in Unicode
```

Sequenze di escape

Per rappresentare caratteri speciali o non stampabili, Java utilizza **sequenze di escape**.
Alcuni esempi comuni:

Sequenza	Nome	Valore Unicode
`\b`	backspace	`\u0008`
`\t`	tabulazione	`\u0009`
`\n`	nuova riga	`\u000a`
`\r`	ritorno carrello	`\u000d`

Sequenza	Nome	Valore Unicode
`\"`	virgolette doppie	`\u0022`
`\'`	virgolette singole	`\u0027`
`\\`	backslash	`\u005c`

Queste sequenze permettono di inserire nel codice caratteri che altrimenti sarebbero difficili da digitare o interpretabili come istruzioni speciali.

Tipo logico (`boolean`)

Il tipo `boolean` rappresenta un valore logico, che può assumere solo due possibili stati:

- `true` **(vero)**
- `false` **(falso)**

Al contrario di `char` o stringhe, i valori booleani **non richiedono apici né virgolette**.

Java

```java
boolean flag = false;
boolean attivo = true;
```

I valori booleani sono utilizzati principalmente nelle **istruzioni di controllo di flusso**, come `if` , `while` e `do` , e sono spesso il risultato di **operatori comparativi**:

Java

```java
int a = 5;
int b = 10;
boolean confronto = (a < b); // true
```

In questo esempio, l'espressione `(a < b)` restituisce un valore booleano (`true`), che può essere usato per decidere l'esecuzione di determinate parti del programma.

Perché le stringhe non sono tra i tipi di dato primitivi in Java

In Java le **stringhe** non sono tipi primitivi, ma **oggetti**.
In molti linguaggi (come C o C++) le stringhe possono essere trattate come tipi primitivi, mentre Sun e Oracle hanno deciso di implementarle come **classi**, per offrire maggiore flessibilità e funzionalità.
Il tipo primitivo di base per i caratteri singoli rimane il `char`.
In linguaggi come Python, invece, tutti i tipi, compresi quelli numerici e le stringhe, sono oggetti, quindi il concetto di tipo primitivo non esiste allo stesso modo.

Dichiarazione delle variabili

In Java **tutte le variabili devono essere dichiarate esplicitamente** prima di poter essere utilizzate.
La dichiarazione di una variabile è composta da due elementi fondamentali:

1. **il tipo:**
   - **che specifica quale genere di dato la variabile può contenere**
2. **l'identificatore:**
   - **cioè il nome della variabile**

Java

```java
int maxValoreCalcolato;
```

In questo esempio:

- `int` è il **tipo**
- `maxValoreCalcolato` è l'**identificatore**

Regole per gli identificatori

Gli identificatori in Java devono rispettare alcune regole sintattiche:

- **sono case-sensitive (** `valore` **e** `Valore` **sono variabili diverse)**
- **possono contenere **lettere, numeri,** `_` **e `$** `**
- **non possono iniziare con un numero**

✓ **Esempi validi:**

```java
int valore1;
int _contatore;
int $somma;
```

× **Esempi non validi:**

```java
int 1valore;   // errore

```

Convenzioni di denominazione

Oltre alle regole obbligatorie, Java segue alcune **convenzioni di stile** per migliorare la leggibilità del codice:

- **l'identificatore inizia sempre con una lettera minuscola**
- **se il nome è composto da più parole, si utilizza la camelCase (prima parola minuscola, le successive con iniziale maiuscola)**

Esempio:

```java
int maxValoreCalcolato;
```

Seguire queste convenzioni non è obbligatorio per il compilatore, ma è una **buona pratica** ampiamente adottata nello sviluppo professionale in Java.

Scope (ambito) delle variabili

In Java, lo **scope** di una variabile rappresenta il suo **ambito di visibilità ed esistenza:**

- **cioè la porzione di codice in cui la variabile è accessibile e può essere utilizzata**.

- **Lo scope è determinato dal blocco di istruzioni delimitato dalle parentesi graffe `{ }` in cui la variabile viene dichiarata**.

Blocchi di istruzioni

I **blocchi di istruzioni** in Java sono delimitati dalle parentesi graffe `{ }` **e servono a raggruppare più istruzioni in un'unica unità logica**.

Le parentesi graffe vengono utilizzate per delimitare:

- **classi**
- **metodi**
- **costrutti di controllo** (`if`, `for`, `while`)
- **blocchi di codice generici**

Java

```java
{
    // blocco di istruzioni
}
```

Ogni blocco di codice definisce un **ambito di visibilità** per le variabili dichiarate al suo interno.
In particolare, una variabile:

- **è visibile solo all'interno del blocco in cui viene dichiarata**
- **rimane utilizzabile anche dopo eventuali blocchi interni**
- **cessa di esistere quando l'esecuzione esce dal suo blocco**

Quando lo scope termina:

- **la variabile non è più accessibile**
- **la memoria associata viene deallocata**

Blocchi annidati e regole sui nomi

In Java **non è consentito dichiarare una variabile con lo stesso nome in un blocco annidato, se esiste già una variabile con quel nome in un blocco esterno**.

Esempio **non valido**:

```java
{
    int a = 10;
    {
        int a = 20; // ERRORE: variabile già dichiarata
    }
}

```

In questo caso, la variabile `a` è già definita nel blocco esterno e non può essere ridefinita in quello interno.

> Nota: in alcuni altri linguaggi di programmazione questo comportamento è permesso, ma **Java lo vieta per evitare ambiguità e rendere il codice più chiaro e sicuro.**

Esempio di scope

```java
{
    statement 1;
    statement 2;
    int a;

    {
        statement 3;
        statement 4;
    }

    statement 5;
    a = 10;   // OK: a è ancora nello scope
    statement 6;
}

System.out.println(a); // ERRORE: a non esiste più
```

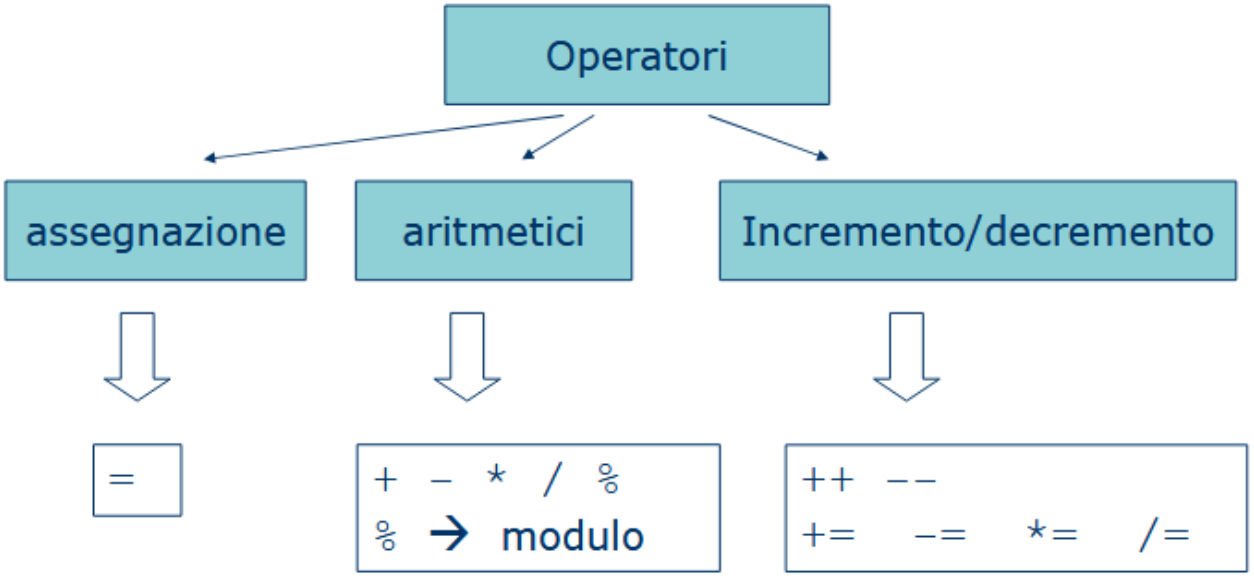
In questo esempio:

- **la variabile `a` è dichiarata nel blocco esterno**
- **può essere utilizzata per tutta la durata del blocco**
- **può essere usata anche dopo un blocco interno**

- **non è accessibile al di fuori del blocco in cui è stata dichiarata**

Operatori in Java

In Java, gli **operatori** servono a manipolare valori e variabili.
Possiamo raggrupparli in varie categorie: **assegnazione, aritmetici e incremento/decremento**.



Operatori di assegnazione

L'operatore di base è `=`:

- **serve ad assegnare un valore a una variabile:**

```java
int a = 5; // assegna 5 alla variabile 'a'
```

È possibile combinare l'assegnazione con operazioni aritmetiche usando operatori abbreviati:

- `+=` → **somma e assegna**
- `-=` → **sottrae e assegna**
- `*=` → **moltiplica e assegna**
- `/=` → **divide e assegna**

Esempio:

```java
int x = 10;
```

```
x += 5; // equivalente a x = x + 5
```

Operatori aritmetici

Gli operatori aritmetici permettono di eseguire operazioni matematiche:

- `+` → **somma**
- `-` → **sottrazione**
- `*` → **moltiplicazione**
- `/` → **divisione**
- `%` → **modulo (resto della divisione)**

> **Nota sulla divisione:**
>
> - Se entrambi gli operandi sono **interi**, il risultato sarà un intero (la parte decimale viene scartata).
> - Se almeno uno degli operandi è in **virgola mobile** (`float` o `double`), il risultato sarà un numero decimale.

Esempi:

Java

```java
int a = 7, b = 2;
int risultato1 = a / b;    // 3
double risultato2 = a / 2.0; // 3.5
int resto = a % b;         // 1
```

Operatori di incremento e decremento

Java fornisce operatori speciali per **aumentare o diminuire il valore di una variabile di 1**:

- `++` : **incremento di 1**
- `--` : **decremento di 1**
- `+=` : **incrementa di un valore a scelta**
- `-=` : **decrementa di un valore a scelta**
- `*=` : **moltiplica con un valore a scelta**

Esempi:

Java

```java
int n = 5;
n++; // equivalente a n = n + 1 → n = 6
n--; // equivalente a n = n - 1 → n = 5
n+= 5; // equivalente a n = n+5= 10
n-=2; // equivalente a n = n-2 = 8
n*=2; // equivalente a n = n*2 = 16
```

Promozione automatica dei tipi

Quando si eseguono operazioni tra **tipi di dati diversi**, **Java effettua automaticamente la promozione al tipo più grande (quello più preciso) tra gli operandi**:

- `int + byte` → `int`
- `float + short` → `float`
- `int + double` → `double`

La scala di precisione dei tipi primitivi è:

Plain text

```
byte < short < int < long < float < double
```

Questo significa che Java converte automaticamente il tipo minore in quello maggiore per evitare perdita di informazione.

⚠ Attenzione:

anche quando gli operandi sono dello stesso tipo, alcune operazioni possono produrre un tipo diverso:

- `byte + byte` → `int`
- `short + short` → `int`
- `int + int` → `int`
- `long + long` → `long`
- `float + float` → `float`
- `double + double` → `double`

Questo comportamento serve a ridurre errori di overflow e garantire che il risultato sia sempre rappresentabile nel tipo più appropriato.

Cast (conversione di tipo)

In Java, a volte è necessario **convertire un valore da un tipo a un altro**.
Questo processo si chiama **cast**.
Esistono due casi principali:

1. **Promozione automatica**:
   - **quando si assegna un tipo "più piccolo" a un tipo "più grande", Java fa la conversione automaticamente, senza richiedere azioni speciali.**
     Esempi:

Java

```java
int a;
int b = 10;
a = b;   // OK: int ← int

int a = 4;
short b = 10;
a = b;   // OK: short → int, promozione automatica
```

2. **Cast esplicito**:
   - **quando si vuole assegnare un tipo "più grande" a uno "più piccolo" o due tipi incompatibili, Java richiede di forzare la conversione, perché potrebbe comportare perdita di dati o overflow**.
     Esempio:

Java

```java
byte a;
int b = 10;

a = b;        // ERRORE: non si può assegnare direttamente
a = (byte)b; // OK: cast esplicito, rischio di perdita di dati
```

Il cast è quindi una **forzatura locale di tipo:**

- **indica al compilatore: “So cosa sto facendo e voglio convertire questo valore”.**
  La sintassi prevede di scrivere il tipo di destinazione tra parentesi prima della variabile:

Java

```java
int x = 10;
byte y = (byte)x;
```

⚠ **Attenzione: il cast può portare a risultati inattesi se il valore originale non è rappresentabile nel tipo di destinazione.**

Esempio con operazioni tra `byte` :

Java

```java
byte a = 18;
byte b = 10;
byte c = a + b;        // ERRORE: a + b viene promosso a int
int c = a + b;         // CORRETTO
byte c2 = (byte)(a+b); // CORRETTO ma attenzione al valore

```

Operatori relazionali

Gli operatori relazionali servono a confrontare due valori e stabilire relazioni di uguaglianza o ordine.
Restituiscono sempre un valore booleano (`true` o `false`).

Operatore	Significato
`==`	uguale a
`!=`	diverso da
`>`	maggiore di
`<`	minore di
`>=`	maggiore o uguale a

Operatore	Significato
`<=`	minore o uguale a

⚠ **Nota importante:**

in Java gli operatori di assegnazione e di confronto sono diversi.

- `=` → **assegna un valore a una variabile**
- `==` → **verifica se due valori sono uguali**

Esempio:

Java

```java
int a = 7;

if (a == 7) {
    System.out.println("a è uguale a 7"); // true
}

// if (a = 7) { ... }  // ERRORE: non compila!
```

ⓘ **Questo controllo evita errori comuni presenti in altri linguaggi in cui l'assegnazione accidentale in un `if` può causare comportamenti inattesi.**

Operatori booleani

Gli operatori booleani permettono di combinare o negare condizioni logiche, restituendo sempre un valore di tipo `boolean` (`true` o `false`).

Operatore	Significato	Note
`||`	OR sc	**Short-circuited**, il secondo operando non viene valutato se il primo è già `false`
`&&`	AND	**Short-circuited**, il secondo operando non viene valutato se il primo è già `false`
`!`	NOT	Negazione logica: Inverte il segno dell'operando da `true` a `false` e

Operatore	Significato	Note
		viceversa
`^`	XOR	OR esclusivo: `true` se i due operandi sono diversi
`|`	OR	Restituisce `true` se uno dei due operandi è `true`
`&`	AND	Restituisce `true` se entrambi gli operandi sono `true`

> **Nota:**
>
> gli operatori **short-circuited** (`&&` , `||`) migliorano l'efficienza evitando valutazioni non necessarie.

Controllo di flusso

Il **controllo di flusso** in Java, come in Python e in JavaScript, permette di modificare l'ordine di esecuzione delle istruzioni in base a condizioni logiche o ripetizioni.
Le principali strutture di controllo includono:

- **Istruzioni condizionali**: `if` , `if-else`, `else if` , `switch`
- **Loop (cicli)**: `for`, `while`, `do-while`
- **Salti indeterminati**: `break`, `continue` , `return`

Istruzione `if-else`

L'istruzione `if` **consente di eseguire un blocco di codice solo se una condizione è vera** .

Java

```java
if (condizione) {
    // istruzioni eseguite solo se la condizione è vera
    istruzione1;
    istruzione2;
}
```

- **La condizione deve sempre essere racchiusa tra parentesi tonde `()` .**
- **Se il blocco contiene una sola istruzione, le parentesi graffe `{}` possono essere omesse.**

- **Il blocco `else` è opzionale e viene eseguito solo se la condizione dell' `if` è falsa o viceversa.**
- **È possibile usare `else if`, come in JavaScript, per gestire più condizioni alternative.**

Esempio:

Java

```java
if (a > 0) {
    System.out.println("a è positivo");
} else if (a < 0) {
    System.out.println("a è negativo");
} else {
    System.out.println("a è zero");
}
```

La logica di questo blocco `if- else if - else` è:

- Se la variabile `a` è maggiore di `0` → stampa il messaggio `"a è positivo"`
- Se la variabile `a` è minore di `0` → stampa il messaggio `"a è negativo"`
- Se la variabile `a` non è né maggiore né minore di `0`; quindi è uguale a `0` (`==`) → stampa il messaggio `"a è zero"`.

Istruzione `switch`

Come in Python con il `match statement` e come nello `switch` di JavaScript, anche Java mette a disposizione l'**istruzione** `switch`,

- **un costrutto di controllo condizionale multiplo.**

Lo `switch` consente di confrontare il valore di una variabile con un insieme di costanti, eseguendo il blocco di codice associato al valore corrispondente.
È particolarmente utile quando si devono gestire **più casi alternativi sulla stessa variabile**, risultando più leggibile ed espressivo rispetto a una lunga catena di `if - else if`.

A differenza di un blocco `if - else if - else`, lo `switch` **non valuta sequenzialmente tutte le condizioni logiche**:

- **il valore dell'espressione viene confrontato direttamente con i `case` disponibili, rendendo il controllo più chiaro dal punto di vista**

semantico.

ⓘ Tipi di variabile ammessi

In Java, la variabile utilizzata nello `switch` può essere di tipo:

- `char`
- `byte`, `short`, `int`
- `enum` (a partire da Java 7)
- `String` (a partire da Java 7)

Non è possibile usare:

- `long`
- tipi floating point (`float`, `double`)
- intervalli di valori

Sintassi di base

Java

```java
int choice = 2;

switch (choice) {
    case 1:
        System.out.println("Hai scelto 1");
        break;
    case 2:
        System.out.println("Hai scelto 2");
        break;
    case 3:
        System.out.println("Hai scelto 3");
        break;
    case 4:
        System.out.println("Hai scelto 4");
        break;
    default:
        System.out.println("Scelta non valida");
}
```

- **Ogni `case` rappresenta un possibile valore della variabile su cui si**

effettua il confronto.

- L'istruzione `break` interrompe l'esecuzione dello `switch`, evitando il cosiddetto **fall-through**.
  - **In assenza di `break`, anche se un `case` corrisponde, tutti i `case` successivi vengono eseguiti in cascata.**
- Il `default` è opzionale e viene eseguito quando nessun `case` corrisponde al valore dell'espressione.

> 🔗 **Nota comparativa**
>
> - In **Python**, il `match` è più flessibile:
> - supporta pattern, strutture e condizioni più complesse.
> - In **JavaScript**, lo `switch` è molto simile a quello di Java e condivide lo stesso comportamento di *fall-through*.
> - In **Java**, lo `switch` è più rigido sui tipi ammessi, ma garantisce maggiore sicurezza statica.

L'istruzione `break`

Come in Python e in JS, lo statement `break` serve interrompere immediatamente il flusso di un ciclo condizionale o di un loop.

All'interno dell'istruzione `switch`, l'uso del `break` è opzionale, ma nella maggior parte dei casi fortemente consigliato.

Il `break`:

- **serve a uscire dallo `switch` una volta eseguito il codice del `case` corrispondente.**
  Se il `break` **non viene inserito**, l'esecuzione **non si ferma** al `case` trovato, ma **prosegue nel `case` successivo**:
- questo comportamento è noto come **fall-through**.

Nella pratica, poiché ogni `case` rappresenta solitamente un caso indipendente, è buona norma inserire un `break` al termine di ciascun `case`.

Fall-through intenzionale

In alcuni casi, **il comportamento di *fall-through* può essere sfruttato volontariamente per eseguire lo stesso codice per più valori distinti della variabile.**
In questo scenario, è possibile **raggruppare più** `case` **consecutivi** e utilizzare **un solo** `break`.
Esempio:

Java

```java
switch (choice) {
    case 1:
    case 2:
        // stesso comportamento per 1 e 2
        istruzioni;
        break;
    case 3:
        istruz3;
        break;
    default:
        // input errato
}
```

Se `choice` **vale** `1`**, l'esecuzione entra nel** `case 1`**, cade nel** `case 2` **e viene eseguito il blocco comune, per poi uscire dallo** `switch`**.**

`switch` come espressione (da Java 12)

A partire da **Java 12**, lo `switch` non è più solo uno *statement*, **ma può essere utilizzato anche come espressione, cioè può restituire un valore.**

In questo caso, il `break` viene utilizzato insieme a un **valore di ritorno**.
Esempio:

Java

```java
String d = switch (day) {
    case "Monday":
        break "Weekday";
    case "Tuesday":
        break "Weekday";
    case "Wednesday":
        break "Weekday";
    case "Thursday":
```

```
9            break "Weekday";
10       case "Friday":
11           break "Weekday";
12       case "Saturday":
13           break "Weekend";
14       case "Sunday":
15           break "Weekend";
16       default:
17           break "unknown";
18   };
```

Poiché lo `switch` valorizza una variabile, è fondamentale prevedere sempre il `default`, per garantire che l'espressione restituisca un valore in ogni caso.

Case multipli (da Java 12)

Sempre da Java 12, **è possibile specificare più valori per uno stesso `case`, separandoli con la virgola**.

Questo rende il codice più compatto e leggibile.

Java

```
1   String s = switch (day) {
2       case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday":
3           break "Weekday";
4       case "Saturday", "Sunday":
5           break "Weekend";
6   };
```

Uso dell'operatore arrow `->` (da Java 12)

Con le versioni più recenti di Java viene introdotto l'operatore **arrow** `->`, già noto per le *lambda expression*.

L'operatore `->`:

- **sostituisce i due simboli `:` e `break`**
- **elimina completamente il rischio di *fall-through***
- **rende lo `switch` più espressivo e sicuro**

Esempio:

```java
String s = switch (day) {
    case "Monday" -> "Weekday";
    case "Tuesday" -> "Weekday";
    case "Wednesday" -> "Weekday";
    case "Thursday" -> "Weekday";
    case "Friday" -> "Weekday";
    case "Saturday" -> "Weekend";
    case "Sunday" -> "Weekend";
    default -> "unknown";
};
```

Con questa sintassi:

- **ogni `case` restituisce direttamente un valore**
- **non è necessario usare `break`**
- **il codice è più simile al `match` di Pythone più sicuro rispetto allo `switch` classico**

> Nota: fino a **Java 13** queste funzionalità erano opzionali; a partire da **Java 14** sono diventate **definitive**.

Loop Indeterminati

In Java, i **loop indeterminati** sono:

- **strutture di controllo che permettono di ripetere un blocco di istruzioni finché una condizione rimane vera, senza sapere a priori quante volte il ciclo verrà eseguito.**

Rientrano in questa categoria i costrutti

- `while`
- `do-while`,
  che si differenziano principalmente per il momento in cui viene valutata la condizione.

Il ciclo `while`

Il ciclo `while` esegue una verifica iniziale della condizione prima di entrare nel blocco di istruzioni.
Sintassi:

Java

```java
while (condizione) {
    // blocco di istruzioni
}
```

Il comportamento è il seguente:

- **la condizione viene valutata prima di ogni iterazione**
- **se la condizione è `false` fin dall'inizio, il blocco non viene mai eseguito**
- **il ciclo continua a iterare **finché la condizione resta `true`**

Confronto con Python e JavaScript

Il ciclo `while` in Java è concettualmente identico a quello presente in Python e JavaScript:

- In Python:

Python

```python
while condizione:
# istruzioni

```

- In JavaScript:

JavaScript

```javascript
while (condizione) {
  // istruzioni
}

```

La differenza principale sta nella **sintassi**: Java richiede parentesi tonde per la condizione e parentesi graffe per il blocco.

Questo tipo di ciclo è utile quando non si conosce in anticipo il numero di iterazioni, ma si sa quando fermarsi.

Il blocco `do - while`

A differenza di Python, che ha il solo ciclo `while`, ma invece molto similarmente a JS(inoltre con una sintassi molto simile a Java),
Il ciclo `do-while` è una variante del `while` che

- **posticipa la verifica della condizione a dopo l'esecuzione del blocco.**

Java

```java
do {
    // blocco di istruzioni
} while (condizione);
```

In questo caso:

- **il blocco di istruzioni viene eseguito almeno una volta**
- **la condizione viene valutata solo al termine dell'iterazione**
- **se la condizione è `true`, il ciclo continua; se è `false`, il ciclo termina**

> **Questo costrutto è utile quando è necessario eseguire almeno una volta un'azione, ad esempio per leggere un input o inizializzare uno stato prima di verificare una condizione.**

Esempi

1. `while`: la condizione è falsa dall'inizio

Java

```java
int x = 10;

while (x < 5) {
    System.out.println("x vale: " + x);
}
```

Cosa succede:

- **la condizione `x < 5` viene valutata prima di entrare nel ciclo**
- **poiché `x` vale `10`, la condizione è false**
- **il blocco non viene mai eseguito**
- **nessuna stampa viene prodotta**

Il ciclo `while` può quindi eseguire il blocco **zero volte**.

2. `do-while` : **la condizione è falsa, ma il blocco viene eseguito**

Java

```java
int x = 10;

do {
    System.out.println("x vale: " + x);
} while (x < 5);
```

Cosa succede:

- il blocco viene eseguito **prima** del controllo della condizione
- viene stampato:

Shell

```
x vale: 10
```

- solo dopo l'esecuzione viene valutata la condizione `x < 5`
- la condizione è `false` , quindi il ciclo termina.

> ⓘ **Il ciclo `do-while` garantisce almeno una esecuzione, anche se la condizione è falsa.**

3. Caso tipico: lettura di input

Java

```java
Scanner sc = new Scanner(System.in);
int numero;

do {
    System.out.print("Inserisci un numero positivo: ");
    numero = sc.nextInt();
} while (numero <= 0);
```

Perché usare `do-while` qui:

- **la richiesta di input deve essere eseguita almeno una volta**

- **la condizione ha senso solo dopo aver letto un valore**
- **il ciclo continua finché l'utente inserisce un numero non valido**

Con un `while`, il codice sarebbe meno naturale.

> **In sintesi**
>
> - usa `while` quando il blocco può anche **non essere eseguito**
> - usa `do-while` quando il blocco **deve essere eseguito almeno una volta**

Loop determinati (`for`)

Dopo aver visto i **loop indeterminati**, in cui il numero di iterazioni **non è noto a priori** e dipende da una condizione (`while`, `do-while`), passiamo ai **loop determinati**.

Un **loop determinato** è:

- **un ciclo in cui il numero di iterazioni è noto o facilmente prevedibile, perché controllato da una variabile di iterazione che segue regole precise.**

In Java, il loop determinato per eccellenza è il `for`.

Struttura del ciclo for

Java

```java
for (inizializzazione; condizione; incremento) {
    // blocco di istruzioni
}
```

Il ciclo `for` è composto da **tre parti fondamentali**:

1. **Inizializzazione**
   - **Viene eseguita una sola volta all'inizio del ciclo**
   - (es. `int i = 0`)
2. **Condizione**
   - **Viene valutata prima di ogni iterazione**
   - **Se è `true`, il ciclo continua; se è `false`, il ciclo termina**

3. **Incremento (o passo)**
   - **Viene eseguito alla fine di ogni iterazione**
   - (es. `i++`, `i += 2`, `i--`)

Esempio base di ciclo `for`

Java

```java
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

Comportamento del ciclo:

- **`i` viene inizializzata a `1`**
- **finché `i <= 10` il ciclo viene eseguito**
- **a ogni iterazione `i` viene incrementata di `1`**
- **vengono stampati i numeri da `1` a `10`**

> La variabile di iterazione (`i`) può essere **dichiarata direttamente nelle parentesi del `for`** ed è visibile **solo all'interno del ciclo**.

Nota

Tornando allo scope delle variabili:
dichiarando una variabile `i` dentro il ciclo `for`, `while` o `do-while` la variabile `i` vive solo dentro il loop
Se si volesse usare la variabile `i` fuori dal for sarebbe impossibile è il compilatore darebbe errore

Confronto con Python e JavaScript

In **Python**, il `for` lavora tipicamente su **collezioni** o **range**:

Python

```python
for i in range(1, 11):
    print(i)

```

In **JavaScript**, il `for` è molto simile a quello di Java:

JS JavaScript

```javascript
for (let i = 1; i <= 10; i++) {
  console.log(i);
}

```

Java utilizza una sintassi più **rigida e tipizzata**, ma concettualmente il funzionamento è analogo.

L'istruzione `continue`

Come per Python e JS l'istruzione `continue`:

- **consente di saltare l'iterazione corrente del ciclo e passare direttamente alla successiva, senza eseguire le istruzioni rimanenti del blocco.**

Java

```java
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        if (i == 12 && j == 5)
            continue;
        System.out.println(j);
    }
    System.out.println(" -- " + i);
}
}
```

In questo esempio:

- **quando** `i == 12` **e** `j == 5`
- **la stampa di** `j` **viene saltata**
- **il ciclo interno continua con** `j = 6`

Il `continue` agisce **solo sul ciclo più interno** in cui compare.

Label-continue

In presenza di cicli annidati, Java consente di usare una etichetta (label) per indicare a quale ciclo applicare il `continue`**.**
L'etichetta deve avere:

- **un nome composto da una sola parola**
- **deve essere seguito da "`:`"**

Java

```java
esterno:
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        if (i == 12 && j == 5)
            continue esterno;
        System.out.println(j);
    }
    System.out.println(" -- " + i);
}

}
```

In questo caso:

- **quando** `i == 12` **e** `j == 5`
- **si salta direttamente all'iterazione successiva del ciclo esterno**
- **non vengono stampati** `j = 5...9`
- **non viene stampato** `i = 12`

Istruzione `break`

Come abbiamo gia visto nell' istruzione `switch`:

- **l'istruzione** `break` **interrompe completamente il ciclo in cui compare.**

Java

```java
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        if (i == 12 && j == 5)
            break;
        System.out.println(j);
    }
    System.out.println(" -- " + i);
}

```

In questo esempio:

- quando `i == 12` e `j == 5`
- il ciclo interno su `j` viene **terminato**
- il ciclo esterno su `i` continua normalmente

`label-break` (break etichettato)

Come per `continue`, anche `break` può essere **etichettato** per interrompere un ciclo esterno.
L'etichetta deve avere:

- **un nome composto da una sola parola**
- **deve essere seguito da " `:` "**

Java

```java
esterno:
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        if (i == 12 && j == 5)
            break esterno;
        System.out.println(j);
    }
    System.out.println(" -- " + i);
}
```

In questo caso:

- **quando `i == 12` e `j == 5`**
- **viene interrotto il ciclo esterno**
- **entrambi i cicli terminano immediatamente**

Passaggio dei parametri in Java

Quando si passa un parametro a un metodo in Java, è fondamentale comprendere **come avviene il passaggio dei valori**, **perché questo influisce direttamente sulle modifiche che un metodo può o non può apportare ai dati del chiamante**.

Passaggio per valore

- **In Java, tutti i parametri vengono sempre passati per valore.**

Per i **tipi primitivi** (`int`, `double`, `char`, `boolean`, ecc.) questo significa che:

- **al metodo viene passata una copia del valore**
- **il metodo lavora solo sulla copia**
- **eventuali modifiche effettuate non sono visibili all'esterno del metodo**

In altre parole, il metodo **non può modificare direttamente** la variabile originale del chiamante.

Esempio di passaggio per valore con tipo primitivo:

Java

```java
public class PassaggioPerValore {

    public static void main(String[] args) {
        int n = 30;
        System.out.println("n vale " + n);   // stampa 30

        nonModifica(n);

        System.out.println("n vale ancora " + n); // stampa 30
    }

    public static void nonModifica(int i) {
        System.out.println("i vale " + i);   // stampa 30
        i = 0;
        System.out.println("adesso i vale " + i); // stampa 0
    }
}
```

Cosa succede passo per passo:

1. **La variabile `n` vale `30`**
2. **`n` viene passata al metodo `nonModifica`**
3. **Il parametro `i` riceve una copia di `n`**
4. **La modifica `i = 0` riguarda solo la copia**
5. **Terminato il metodo, `n` resta invariata**

🔗 Confronto concettuale con Python e JavaScript

- In **Python**, i tipi immutabili (`int` , `float` , `str` , `tuple`) si comportano in modo simile: la modifica all'interno della funzione non influenza il chiamante.

- In **JavaScript**, i tipi primitivi (`number` , `string` , `boolean`) sono anch'essi passati per valore.

Java mantiene una distinzione molto netta e rigorosa, utile per scrivere codice affidabile.

Questo comportamento è coerente e prevedibile ed evita **effetti collaterali indesiderati**.

Stampe e formattazioni

Quando si lavora con l'output standard in Java, spesso `System.out.println()` è sufficiente per stampare valori in modo semplice e immediato.
Tuttavia, quando è necessario controllare il formato dei numeri, in particolare dei valori decimali, è preferibile utilizzare il metodo `printf()`.

Perché usare `printf()`

`printf()` consente di:

- **stabilire quante cifre stampare**
- **controllare il numero di decimali**
- **definire l'allineamento e la precisione**
- **ottenere un output più leggibile e professionale**

Questo è particolarmente utile in:

- report numerici
- log
- stampe tabellari
- applicazioni scientifiche o finanziarie

Struttura del metodo `printf()`

Il metodo `printf()` riceve due elementi fondamentali:

1. **Una stringa di formattazione**
2. **Uno o più valori da formattare**

Java

```java
System.out.printf("stringa di formattazione", valore);
```

La stringa di formattazione contiene dei **segnaposto**, introdotti dal carattere `%`, che indicano **come deve essere stampato il valore**.

Formattazione dei numeri decimali

Tra i formati più utilizzati per i numeri decimali troviamo:

- `%f` → **formato *floating point* (decimale classico)**
- `%g` → **formato *general* (sceglie automaticamente la rappresentazione più compatta)**

Questi formati permettono di controllare la **precisione** del numero stampato.

Specifica della precisione

La precisione si indica **tra il simbolo `%` e la lettera finale (`f` o `g`)**, secondo la forma generale:

```CSS
%X.Yf
%X.Yg
```

Dove:

- `X`:
  - **indica il numero minimo di cifre intere**
- `Y` indica:
  - con `%f`: **il numero di cifre decimali**
  - con `%g`: **il numero totale di cifre significative**

Esempio pratico:

```Java
double x = 12.345;

// stampa senza formattazione
System.out.println(x);

// stampa con formattazione GENERAL
System.out.printf("%3.3g%n", x);

// stampa con formattazione FLOATING POINT
```

```
System.out.printf("%2.2f%n", x);
```

Output prodotto:

- `12.345`
  → stampa senza alcuna formattazione
- `12.3`
  → formato **general** (`%g`) con 3 cifre totali significative
- `12.35`
  → formato **floating point** (`%f`) con 2 cifre decimali
  (il valore viene arrotondato)

Differenza concettuale tra `%f` e `%g`

- `%f`
  → **mantiene sempre la notazione decimale, mostrando esattamente il numero di cifre dopo la virgola richieste**
- `%g`
  → **privilegia una rappresentazione più compatta, limitando il numero totale di cifre e adattando il formato**

> **Nota sull'arrotondamento**
> Quando si riduce il numero di cifre decimali, Java applica automaticamente le **regole di arrotondamento matematico**, rendendo `printf()` affidabile per la presentazione dei dati numerici.